

EXPERIMENT - 6 Snort

IDS/IPS Using TryHackMe

Aim :

To understand and demonstrate the working of Snort as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) by detecting real-time network threats using the TryHackMe platform.

Procedure:

Step 1: Login to Platform

Open TryHackMe in a browser and log in using your account.

Step 2: Open Snort Room

Search for a Snort-related room and join it to begin the experiment.

Step 3: Start AttackBox

Launch the AttackBox, which provides a ready-to-use Linux environment for performing the tasks.

Step 4: Verify Snort Installation

Open the terminal and confirm that Snort is installed and ready to use.

Step 5: Run Snort in IDS Mode

Start Snort in detection mode so that it monitors network traffic and generates alerts for suspicious activity.

Step 6: Detect Network Threats

Perform the tasks given in the room and observe how Snort identifies and reports potential threats in real time.

Step 7: Configure IPS Mode

Enable prevention mode to allow Snort to block malicious traffic based on predefined rules.

Step 8: Analyze Alerts

Review the alerts generated by Snort and identify different types of network attacks.

Output:

[← Back to all walkthroughs](#)



Snort

Learn how to use Snort to detect real-time threats, analyse recorded traffic files and identify anomalies.

   120 min  111,683 

[Share your achievement](#) [Badge](#) [Save Room](#) [2004 Recommend](#) [Options](#)

Room completed (100%)

Task 1  Introduction

Task 2  Interactive Material and VM 

Task 3  Introduction to IDS/IPS

Task 4  First Interaction with Snort

Task 5  Operation Mode 1: Sniffer Mode

Task 6  Operation Mode 2: Packet Logger Mode

Task 7  Operation Mode 3: IDS/IPS

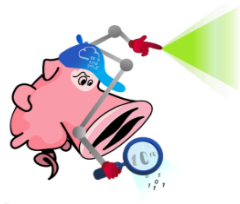
Task 8  Operation Mode 4: PCAP Investigation

Task 9  Snort Rule Structure

Task 10  Snort2 Operation Logic: Points to Remember

Task 1  Introduction

SNORT is an open-source, rule-based Network Intrusion Detection and Prevention System (NIDS/NIPS). It was developed and still maintained by Martin Roesch, open-source contributors, and the Cisco Talos team. The official description: "Snort is the foremost Open Source Intrusion Prevention System (IPS) in the world. Snort IPS uses a series of rules that help define malicious network activity and uses those rules to find packets that match against them and generate alerts for users."



Learning Objectives

In this room, we will cover the following learning objectives:

- Overview of IDS / IPS
- Snort Overview
- Writing IDS rules
- Detecting Intrusion with Snort

Prerequisites

This room expects you to be familiar with basic [Linux](#) command-line functionalities like general system navigation and Network fundamentals (ports, protocols, and traffic data). The room aims to encourage you to start working with Snort to analyse live and captured traffic. Before joining this room, explore the following rooms:

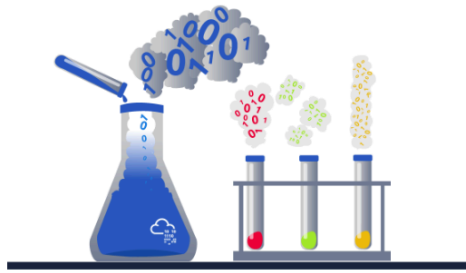
- [Network Fundamentals Module](#).
- [Linux Fundamentals rooms](#) (1 2 3).

Answer the questions below

Read the task above.

No answer needed

 Correct Answer



Deploy the machine attached to this task; once it is ready, it will be visible in the split-screen view. If you don't see a virtual machine load, click the **Show Split View** button.

Set up your virtual environment

To successfully complete this room, you'll need to set up your virtual environment. This involves starting the Target Machine, ensuring you're equipped with the necessary tools and access to tackle the challenges ahead.



Target machine

Status: OFF

[▶ Start Machine](#)

Interactive material and exercise setup

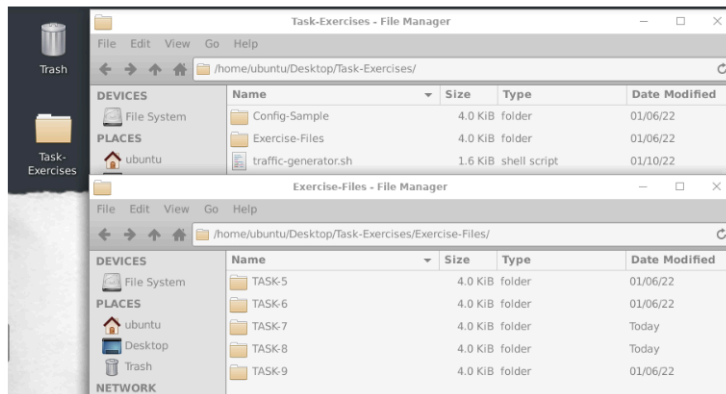
Once the machine has fully started, you will see a folder named **"Task-Exercises"** on the Desktop. Each exercise has an individual folder and files; use them accordingly to the questions.

Everything you need is located under the **"Task-Exercises"** folder.

There are two sub-folders available:

- **Config-Sample** - Sample configuration and rule files. These files are provided to show what the configuration files look like. Installed Snort instance doesn't use them, so feel free

- **Exercise-Files** - There are separate folders for each task. Each folder contains pcap, log, and rule files ready to play with.

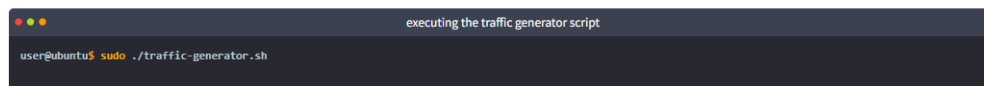


Traffic Generator

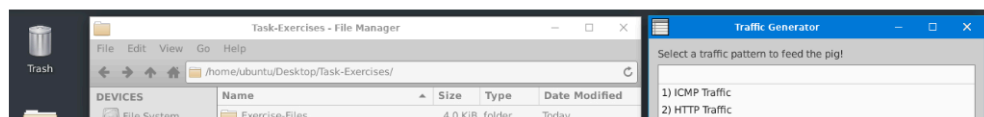
The machine is offline, but there is a script **traffic-generator.sh** for you to generate traffic to your Snort interface. You will use this script to trigger traffic to the Snort interface. Once you run the script, it will ask you to choose the exercise type and then automatically open another terminal to show you the output of the selected action.

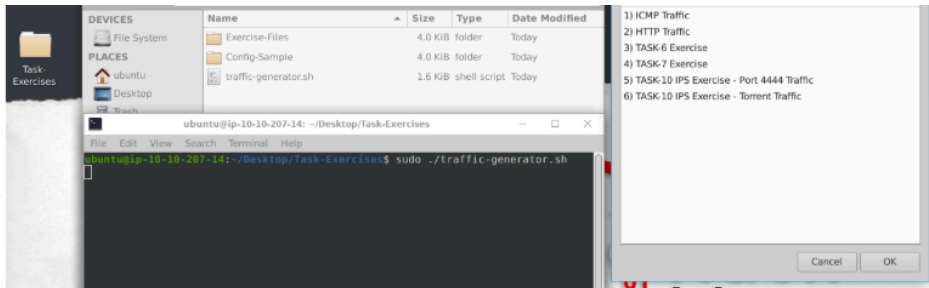
Note that each traffic flow is designed for a specific exercise. Make sure you start the Snort instance and wait until the end of the script execution. Don't stop the traffic flow unless you choose the wrong exercise.

Run the **"traffic generator.sh"** file by executing it as **sudo**.

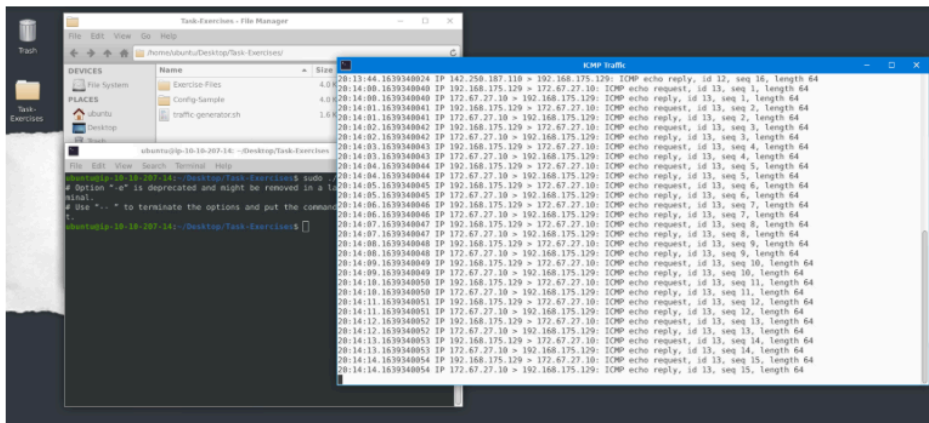


General desktop overview. Traffic generator script in action.





Once you choose an action, the menu disappears, and a terminal instance opens to show you its output.



Answer the questions below

Navigate to the Task-Exercises folder and run the command './easy.sh' and write the output

Too Easy!

✓ Correct Answer

Task 3 Introduction to IDS/IPS



Before diving into Snort and analysing traffic, let's briefly overview what an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) are. You can configure your network infrastructure to use both, but before starting to use any of them, let's learn the differences.

Intrusion Detection System (IDS)

IDS is a passive monitoring solution for detecting possible malicious activities/patterns, abnormal incidents, and policy violations. It generates alerts for each suspicious event.

There are two main types of IDS systems:

- **Network Intrusion Detection System (NIDS):** NIDS monitors the traffic flow from various areas of the network. The aim is to investigate the traffic on the entire subnet. If a signature is identified, an alert is created.
- **Host-based Intrusion Detection System (HIDS):** HIDS monitors the traffic flow from a single endpoint device. Its aim is to investigate the traffic on that device. If a signature is identified, an alert is created.

Intrusion Prevention System (IPS)

IPS is an active protecting solution for preventing possible malicious activities/patterns, abnormal incidents, and policy violations. It is responsible for stopping/preventing/terminating the suspicious event as soon as it is detected.

There are four main types of IPS systems;

- **Network Intrusion Prevention System (NIPS):** NIPS monitors the traffic flow from various areas of the network. The aim is to protect the traffic on the entire subnet. If a signature

- **Behaviour-based Intrusion Prevention System (Network Behaviour Analysis - NBA):** Behaviour-based systems monitor the traffic flow from various areas of the network. The aim is to protect the traffic on the entire subnet. If an anomaly is identified, the connection is terminated.

The Network Behaviour Analysis System works similar to NIPS. The difference between NIPS and Behaviour-based systems is that behaviour-based systems require a training period (also known as "baselining") to learn the normal traffic and differentiate the malicious traffic and threats. This model provides more efficient results against new and unknown threats.

The system is trained to know the "normal" to detect the "abnormal." The training period is crucial to avoid false positives. If there is a security breach during the training period, the results will be highly problematic. Another critical point is to ensure that the system is well trained to recognise benign activities.

- **Wireless Intrusion Prevention System (WIPS):** WIPS monitors the traffic flow from a wireless network. Its aim is to protect wireless traffic and stop possible attacks launched from there. If a signature is identified, the connection is terminated.
- **Host-based Intrusion Prevention System (HIPS):** HIPS actively protects the traffic flow from a single endpoint device. The aim is to investigate the traffic on a particular device. If a signature is identified, the connection is terminated.

The working mechanism of HIPS is similar to that of HIDS. The difference is that while HIDS creates alerts for threats, HIPS stops them by terminating the connection.

Detection/Prevention Techniques

There are three main detection and prevention techniques used in IDS and IPS solutions;

Technique	Approach
Signature-Based	This technique relies on rules that identify the specific patterns of the known malicious behaviour. This model helps detect known threats.
Behaviour-Based	This technique identifies new threats with new patterns that pass through signatures. The model compares known/normal behaviours with unknown/abnormal behaviours, helping detect previously unknown or new threats.
Policy-Based	This technique compares detected activities with system configuration and security policies. This model helps detect policy violations.

Summary

Let's summarise the overall functions of the IDS and IPS in a nutshell.

- IDS can identify threats but requires user assistance to stop them.
- IPS can identify and block threats with less user assistance at the time of detection.

Snort

Now let's talk about Snort. [Here is the rest of the official description](#) of Snort;

"Snort can be deployed in-line to stop these packets, as well. Snort has three primary uses: As a packet sniffer like tcpdump, as a packet logger, which is useful for network traffic debugging or can be used as a full-blown network intrusion prevention system. Snort can be downloaded and configured for personal and business use alike."

SNORT is an open-source, rule-based Network Intrusion Detection and Prevention System (NIDS/NIPS). It was developed and still maintained by Martin Roesch, open-source contributors, and the Cisco Talos team.

Capabilities of Snort:

Snort has the following capabilities:

- Live traffic analysis
- Attack and probe detection
- Packet logging
- Protocol analysis
- Real-time alerting
- Modules & plugins
- Pre-processors
- Cross-platform support! ([Linux](#) & [Windows](#))

Snort Modes

Snort has the following three main use modes:

- **Sniffer Mode:** Read and prompt IP packets in the console application.
- **Packet Logger Mode:** Log all IP packets (inbound and outbound) that visit the network.
- **NIDS (Network Intrusion Detection System) and NIPS (Network Intrusion Prevention System) Modes:** Log/drop the packets deemed malicious according to the user-defined rules.

Answer the questions below

Which IDS or IPS type can help you stop the threats on a local machine?

HIPS

✓ Correct Answer

Which IDS or IPS type can help you detect threats on a local network?

NIDS

✓ Correct Answer

Which IDS or IPS type can help you detect the threats on a local machine?

HIDS

✓ Correct Answer

Which IDS or IPS type can help you stop the threats on a local network?

NIPS

✓ Correct Answer

Which described solution works by detecting anomalies in the network?

NBA

✓ Correct Answer

According to the official description of the snort, what kind of NIPS is it?

full-blown

✓ Correct Answer

NBA training period is also known as ...

baselining

✓ Correct Answer

Task 4 First Interaction with Snort

The First Interaction with Snort

First, let's verify Snort is installed. The following command will show you the instance version.

```
version check

user@ubuntu:~$ snort -V

_ _ _ _ _
o" )~ Version 2.9.7.0 GRE (Build XXXXXX)
' ' ' ' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
        Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
        Copyright (C) 1998-2013 Sourcefire, Inc., et al.
        Using libpcap version 1.9.1 (with TPACKET_V3)
        Using PCRE version: 8.39 2016-06-14
        Using ZLIB version: 1.2.11
```

Before getting your hands dirty, we should ensure our configuration file is valid.

Here, "-T" is used to test the configuration, and "-c" identifies configuration file (**snort.conf**).

Note that it is possible to use an additional configuration file by pointing to it with "-c".

```
Configuration check

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -T

==== Initializing Snort ====
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
... [Output truncated]
==== Initialization Complete ====

_*> Snort! <*-
o" )~ Version 2.9.7.0 GRE (Build XXXX)
''' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
    Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
    Copyright (C) 1998-2013 Sourcefire, Inc., et al.
    Using libpcap version 1.9.1 (with TPACKET_V3)
    Using PCRE version: 8.39 2016-06-14
    Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4
Preprocessor Object: SF_GTP Version 1.1
Preprocessor Object: SF_SIP Version 1.1
Preprocessor Object: SF_SSH Version 1.1
Preprocessor Object: SF_SMTP Version 1.1
Preprocessor Object: SF_POP Version 1.0
Preprocessor Object: SF_DCERPC2 Version 1.0
Preprocessor Object: SF_IMAP Version 1.0
Preprocessor Object: SF_DNP3 Version 1.1
Preprocessor Object: SF_SSLPP Version 1.1
Preprocessor Object: SF_MODBUS Version 1.1
Preprocessor Object: SF_SDF Version 1.1
Preprocessor Object: SF_REPUTATION Version 1.1
Preprocessor Object: SF_DNS Version 1.1
Preprocessor Object: SF_FTPTELNET Version 1.2
... [Output truncated]
Snort successfully validated the configuration!
Snort exiting
```

Once we use a configuration file, Snort has much more power! The configuration file is Snort's all-in-one management file. Rules, plugins, detection mechanisms, default actions, and output settings are identified here. Multiple configuration files are possible for different purposes and cases, but we can only use one at runtime.

Note that Snort automatically shows the default banner and initial information about your setup when you start it. You can prevent this by using the "-q" parameter.

Parameter	Description
-V / --version	This parameter provides information about your instance version.
-c	Identifying the configuration file
-T	Snort's self-test parameter allows you to test your setup with this parameter.
-q	Quiet mode prevents Snort from displaying the default banner and initial information about your setup.

That was easy; let's move on to the next task and continue exploring snort modes.

Answer the questions below

Run the Snort instance and check the build number.

149

✓ Correct Answer



Test the current instance with "/etc/snort/snort.conf" file and check how many rules are loaded with the current build.

4151

✓ Correct Answer

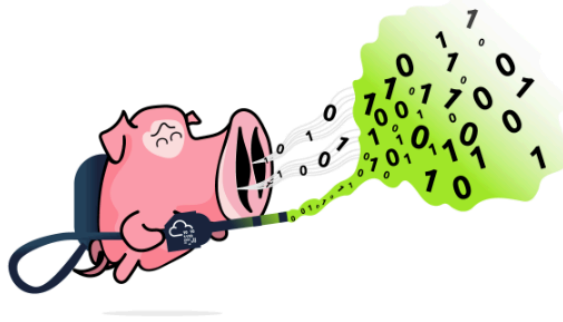


Test the current instance with "/etc/snort/snortv2.conf" file and check how many rules are loaded with the current build.

1

✓ Correct Answer





Let's run Snort in Sniffer Mode.

Like tcpdump, Snort has various flags that allow for viewing different data about the packet it is ingesting.

Sniffer mode parameters are explained in the table below;

Parameter	Description
-v	Verbose. Display the <u>T</u> C <u>P</u> / <u>I</u> P output in the console.
-d	Display the packet data (payload).
-e	Display the link-layer (<u>T</u> C <u>P</u> / <u>I</u> P/ <u>U</u> D <u>P</u> / <u>I</u> C <u>M</u> P) headers.
-X	Display the full packet details in HEX.
-i	This parameter helps define a specific network interface to listen to or sniff. Once you have multiple interfaces, you can choose a specific interface to sniff.

Let's start using each parameter and see the difference between them. Snort needs active traffic on your interface, so we need to generate traffic to see Snort in action.

To do this, use **the traffic-generator script** (find this in the Task-Exercise folder).

Sniffing with parameter "-i"

Start the Snort instance in **verbose mode (-v)** and **use the interface (-i) "eth0"**; `sudo snort -v -i eth0`

In case you have only one interface, Snort uses it by default. The above example demonstrates to sniff on the interface named "eth0". Once you simulate the parameter -v, you will notice it will automatically use the "eth0" interface and prompt it.

Sniffing with parameter "-v"

Start the Snort instance in **verbose mode (-v)**; `sudo snort -v`

Now run the traffic-generator script as sudo and start **ICMP/HTTP traffic**. Once the traffic is generated, snort will start showing the packets in verbosity mode as follows;

```
sniffing with -v

user@ubuntu$ sudo snort -v

Running in packet dump mode

==== Initializing Snort ====
...
Commencing packet processing (pid=64)
12/01-20:10:13.846653 192.168.175.129:34316 -> 192.168.175.2:53
UDP TTL:64 TOS:0x0 ID:23826 IpLen:20 DgmLen:64 DF
Len: 36
=====

12/01-20:10:13.846794 192.168.175.129:38655 -> 192.168.175.2:53
UDP TTL:64 TOS:0x0 ID:23827 IpLen:20 DgmLen:64 DF
Len: 36
=====
Snort exiting
```

As you can see in the given output, verbosity mode provides tcpdump-like output information. Once we interrupt the sniffing with CTRL+C, it stops and summarizes the sniffed packets.

Sniffing with parameter "-d"

Start the Snort instance in **dumping packet data mode (-d)**; `sudo snort -d`

Now run the traffic-generator script as **sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, snort will start showing the packets in verbosity mode as follows;

```
sniffing with -d

user@ubuntu$ sudo snort -d

Running in packet dump mode

--== Initializing Snort ==--
...
Commencing packet processing (pid=67)

12/01-20:45:42.068675 192.168.175.129:37820 -> 192.168.175.2:53
UDP TTL:64 TOS:0x0 ID:53099 IpLen:20 DgmLen:56 DF
Len: 28
99 A5 01 00 00 01 00 00 00 00 00 06 67 6F 6F .....goo
67 6C 65 03 63 6F 6D 00 00 1C 00 01          gle.com.....

=====

WARNING: No preprocessors configured for policy 0.
12/01-20:45:42.070742 192.168.175.2:53 -> 192.168.175.129:44947
UDP TTL:128 TOS:0x0 ID:63307 IpLen:20 DgmLen:72
Len: 44
FE 64 81 80 00 01 00 01 00 00 00 06 67 6F 6F .d.....goo
67 6C 65 03 63 6F 6D 00 00 01 00 01 C0 0C 00 01 gle.com.....
00 01 00 00 00 05 00 04 D8 3A CE CE          .....!..

=====
```

As you can see in the given output, packet data payload mode covers the verbose mode and provides more data.

Sniffing with parameter "-de"

Start the Snort instance in **dump (-d) and link-layer header grabbing (-e)** mode; `snort -d -e`

Now run the traffic-generator script as **sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, snort will start showing the packets in verbosity mode as follows;

```
sniffing with -de

user@ubuntu$ sudo snort -de

Running in packet dump mode

--== Initializing Snort ==--
...
Commencing packet processing (pid=70)
12/01-20:55:26.958773 00:0C:29:A5:87:A2 -> 00:50:56:E1:9B:9D type:0x800 len:0x46
192.168.175.129:47395 -> 192.168.175.2:53 UDP TTL:64 TOS:0x0 ID:64294 IpLen:20 DgmLen:56 DF
Len: 28
6D 9C 01 00 00 01 00 00 00 00 00 06 67 6F 6F m.....goo
67 6C 65 03 63 6F 6D 00 00 01 00 01          gle.com.....

=====

WARNING: No preprocessors configured for policy 0.
12/01-20:55:26.965226 00:50:56:E1:9B:9D -> 00:0C:29:A5:87:A2 type:0x800 len:0x56
192.168.175.2:53 -> 192.168.175.129:47395 UDP TTL:128 TOS:0x0 ID:63346 IpLen:20 DgmLen:72
Len: 44
6D 9C 81 80 00 01 00 01 00 00 00 06 67 6F 6F m.....goo
67 6C 65 03 63 6F 6D 00 00 01 00 01 C0 0C 00 01 gle.com.....
00 01 00 00 00 05 00 04 D8 3A D6 8E          .....!..
```

Sniffing with parameter "-X"

Start the Snort instance in **full packet dump mode (-X)**; `sudo snort -X`

Now run the traffic-generator script as **sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, snort will start showing the packets in verbosity mode as follows;

```
sniffing with -X
```

```
user@ubuntu$ sudo snort -X
```

```
Running in packet dump mode
```

```
--== Initializing Snort ==--
```

```
...
```

```
Commencing packet processing (pid=76)
```

```
WARNING: No preprocessors configured for policy 0.
```

```
12/01-21:07:56.806121 192.168.175.1:58626 -> 239.255.255.250:1900
```

```
UDP TTL:1 TOS:0x0 ID:48861 IpLen:20 DgmLen:196
```

```
Len: 168
```

```
0x0000: 01 00 5E 7F FF FA 00 50 56 C0 00 08 08 00 45 00 ..^....PV.....E.
```

```
0x0010: 00 CA BE DD 00 00 01 11 9A A7 C0 A8 AF 01 EF FF .....M-SEAR
```

```
0x0020: FF FA E5 02 07 6C 00 B0 85 AE 4D 2D 53 45 41 52 .....CH * HTTP/1.1..H
```

```
0x0030: 43 48 20 2A 20 48 54 54 50 2F 31 2E 31 0D 0A 48 OST: 239.255.255
```

```
0x0040: 4F 53 54 3A 20 32 33 39 2E 32 35 35 2E 32 35 35 .250:1900..MAN:
```

```
0x0050: 2E 32 35 30 3A 31 39 30 30 0D 0A 4D 41 4E 3A 20 "ssdp:discover".
```

```
0x0060: 22 73 64 70 3A 64 69 73 63 6F 76 65 72 22 0D .MX: 1..ST: urn:
```

```
0x0070: 0A 4D 58 3A 20 31 0D 0A 53 54 3A 20 75 72 6E 3A dial-multiscreen
```

```
0x0080: 64 69 61 6C 2D 6D 75 6C 74 69 73 63 72 65 65 6E -org:service:dia
```

```
0x0090: 2D 6F 72 67 3A 73 65 72 76 69 63 65 3A 64 69 61 1:1..USER-AGENT:
```

```
0x00A0: 6C 3A 31 0D 0A 55 53 45 52 2D 41 47 45 4E 54 3A Chromium/95.0.4
```

```
0x00B0: 20 43 68 72 6F 6D 69 75 6D 2F 39 35 2E 30 2E 34 638.69 Windows..
```

```
0x00C0: 36 33 38 2E 36 39 20 57 69 6E 64 6F 77 73 0D 0A ..
```

```
0x00D0: 0D 0A ..
```

```
..
```

```
+++++
```

```

WARNING: No preprocessors configured for policy 0.
12/01-21:07:57.624205 216.58.214.142 -> 192.168.175.129
ICMP TTL:128 TOS:0x0 ID:63394 IpLen:20 DgmLen:84
Type:0 Code:0 ID:15 Seq:1 ECHO REPLY
0x0000: 00 0C 29 A5 B7 A2 00 50 56 E1 9B 9D 08 00 45 00  ..)...PV.....E.
0x0010: 00 54 F7 A2 00 00 80 01 24 13 D8 3A D6 8E C0 A8  .T.....$......
0x0020: AF 81 00 00 BE B6 00 0F 00 01 2D E4 A7 61 00 00  .....-..a...
0x0030: 00 00 A4 20 09 00 00 00 00 00 10 11 12 13 14 15  ...
0x0040: 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 24 25  .....!"#$%
0x0050: 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35  &'()*+,-./012345
0x0060: 36 37                                     67

=====

```

Note that you can use the parameters both in combined and separated form as follows;

- snort -v
- snort -vd
- snort -de
- snort -v -d -e
- snort -X

Make sure you understand and practice each parameter with different types of traffic and discover your favourite combination.

Answer the questions below

You can practice the parameter combinations by using the traffic-generator script.

No answer needed

✓ Correct Answer



Let's run Snort in Logger Mode

You can use Snort as a sniffer and log the sniffed packets via logger mode. You only need to use the packet logger mode parameters, and Snort does the rest to accomplish this.

Packet logger parameters are explained in the table below;

Parameter	Description
-l	Logger mode, target log, and alert output directory. Default output folder is /var/log/snort The default action is to dump as tcpdump format in /var/log/snort
-K ASCII	Log packets in ASCII format.
-r	Reading option: Review the logged events in Snort.
-n	Specify the number of packets to be processed or read. Snort will stop after reading the specified number of packets.

Let's start using each parameter and see the difference between them. Snort needs active traffic on your interface, so we need to generate traffic to see Snort in action.

Logfile Ownership

Before generating logs and investigating them, we must remember the **Linux** file ownership and permissions. There is no need to delve into user types and permissions. The fundamental file ownership rule:

Snort needs superuser (root) rights to sniff the traffic, so once you run Snort with the "sudo" command, the "root" account will own the generated log files. Therefore, you will need "root" rights to investigate the log files. There are two different approaches to analyzing the generated log files:

Elevation of privileges

- Changing the ownership of files/directories - You can also change the ownership of the file/folder to read it as your user: `sudo chown username file` or `sudo chown username -R directory.` The "-R" parameter enables recursive processing of files and directories.

Logging with parameter "-l"

First, start the Snort instance in packet logger mode: `sudo snort -dev -l.`

Now start ICMP/HTTP traffic with the traffic-generator script.

Once the traffic is generated, Snort will start showing the packets and log them in the target directory. You can configure the default output directory in Snort.config file. However, you can use the "-l" parameter to set a target directory. Identifying the default log directory is useful for continuous monitoring operations, and the "-l" parameter is much more useful for testing purposes.

The `-l.` part of the command creates the logs in the current directory. You will need to use this option to have the logs for each exercise in their folder.

```
logging with -l

user@ubuntu$ sudo snort -dev -l.

Running in packet logging mode

---= Initializing Snort ==--
Initializing Output Plugins!
Log directory = /var/log/snort
pcap DAQ configured to passive.
Acquiring network traffic from "ens33".
Decoding Ethernet

---= Initialization Complete ==--
...
Commencing packet processing (pid=2679)
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
```

Now, let's check the generated log file. **Note that the log file names will be different in your case.**

```
Checking the log file

user@ubuntu$ ls.
snort.log.1638459842
```

As you can see, it is a single all-in-one log file. It is a binary/tcpdump format log. This is what it looks like in the folder view.

Name	Size
 snort.log.1638459842	6.3 kB

Logging with parameter "-K ASCII"

Start the Snort instance in packet logger mode: `sudo snort -dev -K ASCII`

Now run the traffic-generator script as sudo and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will start showing the packets in verbosity mode as follows:

```
Logging with -K ASCII

user@ubuntu$ sudo snort -dev -K ASCII -l.

Running in packet logging mode

---= Initializing Snort ==--
Initializing Output Plugins!
Log directory = /var/log/snort
pcap DAQ configured to passive.
Acquiring network traffic from "ens33".
Decoding Ethernet

---= Initialization Complete ==--
...
Commencing packet processing (pid=2679)
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
```

Now, let's check the generated log file.

```
Checking the log file

user@ubuntu$ ls .
142.250.187.110 192.168.175.129 snort.log.1638459842
```

This is what it looks like in the folder view.

Name	Size
 142.250.187.110	1 item
 192.168.175.129	5 items

The logs created with the "-K ASCII" parameter are entirely different. There are two folders with IP address names. Let's look into them.

```

checking the log file

user@ubuntu$ ls ./192.168.175.129/

ICMP_ECHO  UDP:36648-53  UDP:40757-53  UDP:47404-53  UDP:50624-123

```

Once we look closer at the created folders, we can see that the logs are in ASCII and categorised format, so it is possible to read them without using a Snort instance.

This is what it looks like in the folder view.

Name	Size
ICMP_ECHO	11.8 kB
UDP:36648-53	1.4 kB
UDP:40757-53	1.2 kB
UDP:47404-53	1.2 kB
UDP:50624-123	911 bytes

In a nutshell, ASCII mode provides multiple files in a human-readable format, allowing for easy reading of the logs using a text editor. In contrast to the ASCII format, the binary format is not human-readable and requires analysis using Snort or a similar application, such as tcpdump.

Let's compare the ASCII format with the binary format by opening both of them in a text editor. The difference between the binary log file and the ASCII log file is shown below. (Left side: binary format. Right side: ASCII format.

[illegible]

Reading generated logs with parameter "-r"

Start the Snort instance in packet reader mode; `sudo snort -r`

```
reading log files with -r

user@ubuntu$ sudo snort -r snort.log.1638459842

Running in packet dump mode

==== Initializing Snort ====
Initializing Output Plugins!
pcap DAQ configured to read-file.
Acquiring network traffic from "snort.log.1638459842".

==== Initialization Complete ===

...
Commencing packet processing (pid=3012)
WARNING: No preprocessors configured for policy 0.
12/02-07:44:03.123225 192.168.175.129 -> 142.250.187.110
ICMP TTL:64 TOS:0x0 ID:41900 IpLen:20 DgmLen:84 DF
Type:8 Code:0 ID:1 Seq:49 ECHO
=====
WARNING: No preprocessors configured for policy 0.
12/02-07:44:26.169620 192.168.175.129 -> 142.250.187.110
ICMP TTL:64 TOS:0x0 ID:44765 IpLen:20 DgmLen:84 DF
Type:8 Code:0 ID:1 Seq:72 ECHO
=====
Packet I/O Totals:
  Received:      51
  Analyzed:      51 (100.000%)
  Dropped:       0 ( 0.000%)
  Filtered:      0 ( 0.000%)
  Outstanding:   0 ( 0.000%)
  Injected:      0
=====
Breakdown by protocol (includes rebuilt packets):
...
  Total:         51
=====
Snort exiting
```

Note that Snort can read and handle binary log output (tcpdump and Wireshark can also handle this log format). However, if you create logs with the "-K ASCII" parameter, Snort will not read them. As you can see in the output above, Snort read and displayed the log file, just as it does in sniffer mode.

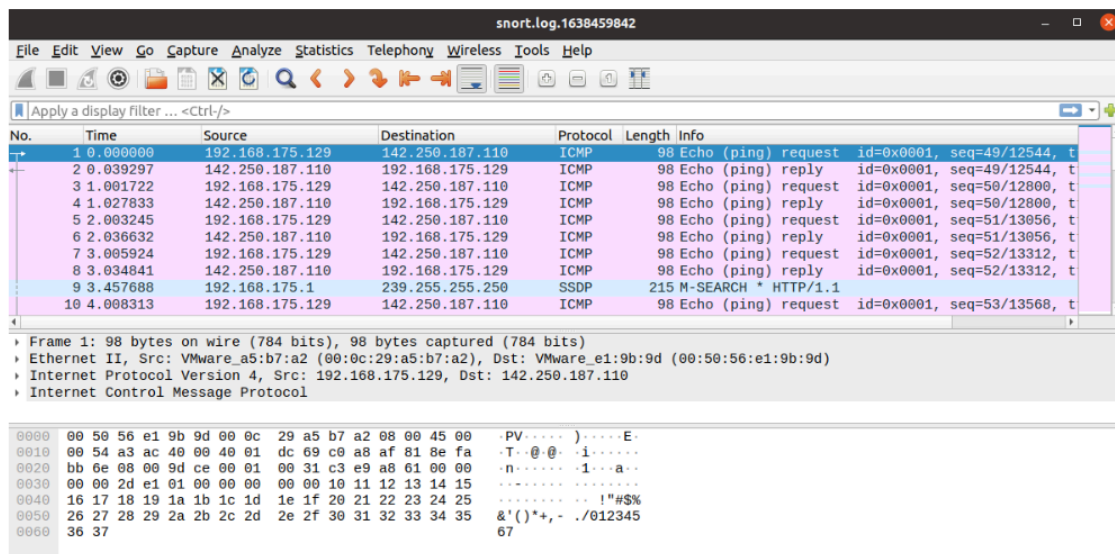
Opening log file with tcpdump.

```
Opening the log file with tcpdump

user@ubuntu$ sudo tcpdump -r snort.log.1638459842 -ntc 10

reading from file snort.log.1638459842, link-type EN10MB (Ethernet)
IP 192.168.175.129 > 142.250.187.110: ICMP echo request, id 1, seq 49, length 64
IP 142.250.187.110 > 192.168.175.129: ICMP echo reply, id 1, seq 49, length 64
IP 192.168.175.129 > 142.250.187.110: ICMP echo request, id 1, seq 50, length 64
IP 142.250.187.110 > 192.168.175.129: ICMP echo reply, id 1, seq 50, length 64
IP 192.168.175.129 > 142.250.187.110: ICMP echo request, id 1, seq 51, length 64
IP 142.250.187.110 > 192.168.175.129: ICMP echo reply, id 1, seq 51, length 64
IP 192.168.175.129 > 142.250.187.110: ICMP echo request, id 1, seq 52, length 64
IP 142.250.187.110 > 192.168.175.129: ICMP echo reply, id 1, seq 52, length 64
IP 192.168.175.1.63096 > 239.255.255.250.1900: UDP, length 173
IP 192.168.175.129 > 142.250.187.110: ICMP echo request, id 1, seq 53, length 64
```

Opening the log file with Wireshark.



The "-r" parameter also allows users to filter the binary log files. You can filter the processed log to see specific packets with the "-r" parameter and Berkeley Packet Filters (BPF).

- `sudo snort -r logname.log -X`
- `sudo snort -r logname.log icmp`
- `sudo snort -r logname.log tcp`
- `sudo snort -r logname.log 'udp and port 53'`

The output will be the same as the above, but only packets with the chosen protocol will be shown. Additionally, you can specify the number of processes using the "-n" parameter. **The following command will process only the first 10 packets:**

```
Snort -dvr logname.log -n 10
```

Please use the following resources to understand how the [BPF](#) works and its use.

- https://en.wikipedia.org/wiki/Berkeley_Packet_Filter
- <https://biot.com/capstats/bpf.html>
- <https://www.tcpdump.org/manpages/tcpdump.1.html>

Now, use the attached VM and navigate to the Task-Exercises/Exercise-Files/TASK-6 folder to answer the questions!

Answer the questions below

Investigate the traffic with the default configuration file **with ASCII mode**.

```
sudo snort -dev -K ASCII -l .
```

Execute the traffic generator script and choose **"TASK-6 Exercise"**. Wait until the traffic ends, then stop the Snort instance. Now analyse the output summary and answer the question.

```
sudo ./traffic-generator.sh
```

Now, you should have the logs in the current directory. Navigate to folder **"145.254.160.237"**. What is the source port used to connect port 53?

✓ Correct Answer

Use **snort.log.1640048004**

Read the snort.log file with Snort; what is the IP ID of the 10th packet?

```
snort -r snort.log.1640048004 -n 10
```

✓ Correct Answer

Read the **"snort.log.1640048004"** file with Snort; what is the referer of the 4th packet?

✓ Correct Answer

Read the **"snort.log.1640048004"** file with Snort; what is the Ack number of the 8th packet?

✓ Correct Answer

Read the **"snort.log.1640048004"** file with Snort; what is the number of the **"TCP port 80"** packets?

✓ Correct Answer



Snort in IDS/IPS Mode

The capabilities of Snort are not limited to sniffing and logging traffic. IDS/IPS mode enables you to manage traffic according to user-defined rules.

Note that (N)IDS/IPS mode depends on the rules and configuration. **TASK-10** summarises the essential paths, files, and variables. Also, **TASK-3** covers configuration testing. First, we need to understand the operating logic, and then we will proceed to the rules in **TASK-9**.

Let's run Snort in IDS/IPS Mode

NIDS mode parameters are explained in the table below;

Parameter	Description
-c	Defining the configuration file.
-T	Testing the configuration file.
-N	Disable logging.
-D	Background mode.
-A	Alert modes; Full: Full alert mode, providing all possible information about the alert. This mode is also the default; once you use -A and don't specify a mode, Snort defaults to this mode. Fast mode displays the alert message, timestamp, source and destination IP addresses , along with port numbers. Console: Provides fast style alerts on the console screen. cmg: CMG style, basic header details with payload in hex and text format. None: Disabling alerting.

Let's start using each parameter and see the difference between them. Snort needs active traffic on your interface, so we need to generate traffic to see Snort in action. To do this, use **the traffic-generator** script and sniff the traffic.

Once you start running IDS/IPS mode, you need to use rules. As we mentioned earlier, we will use a pre-defined ICMP rule as an example. The defined rule will only generate alerts in any direction of ICMP packet activity.

```
alert icmp any any <> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;)
```

This rule is located in `/etc/snort/rules/local.rules`.

Remember, in this module, we will focus only on the operating modes. The rules are covered in TASK9&10. **Snort will create an "alert" file if the traffic flow triggers an alert. One last note:** once you start running IPS/IDS mode, the sniffing and logging mode will be semi-passive. However, you can activate the functions using the parameters discussed in previous tasks. (-i, -v, -d, -e, -X, -l, -K ASCII) If you don't remember the purpose of these commands, **please refer back to TASK 4**.

IDS/IPS mode with parameter "-c and -T"

Start the Snort instance and test the configuration file. `Sudo snort -c /etc/snort/snort.conf -T` This command will check your configuration file and prompt you if there is any misconfiguration in your current settings. You should be familiar with this command if you covered TASK3. If you don't remember the output of this command, **please revisit TASK4**.

IDS/IPS mode with parameter "-N"

Start the Snort instance and disable logging by running the following command: `sudo snort -c /etc/snort/snort.conf -N`

Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**. This command will disable logging mode. The rest of the other functions will still be available (if activated).

The command-line output will provide the requested information along with the specified parameters. So, if you activate **verbosity (-v)** or **full packet dump (-X)**, you will still have the output in the console, but there will be no logs in the log folder.

IDS/IPS mode with parameter "-D"

Start the Snort instance in background mode with the following command: `sudo snort -c /etc/snort/snort.conf -D`

Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will begin processing the packets and accomplish the given task with additional parameters.

```
Running in background mode

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -D

Spawning daemon child...
My daemon child 2898 lives...
Daemon parent exiting (0)
```

The command-line output will provide the requested information along with the specified parameters. So, if you activate **verbosity (-v)** or **full packet dump (-X)** with **packet logger mode (-l)**, you will still have the logs in the logs folder, but there will be no output in the console.

Once you start the background mode and want to check the corresponding process, you can easily use the "ps" command as shown below;

```
running in background mode

user@ubuntu$ ps -ef | grep snort

root      2898      1706  0 05:53 ?        00:00:00 snort -c /etc/snort/snort.conf -D
```

To stop the daemon, you can use the "kill" command to terminate the process.

```
running in background mode

user@ubuntu$ sudo kill -9 2898
```

Note that daemon mode is mainly used to automate Snort. This parameter is primarily used in scripts to start the Snort service in the background. It is not recommended to use this

IDS/IPS mode with parameter "-A"

Note that Snort offers several alert modes.

- **console:** Provides fast style alerts on the console screen.
- **cmg:** Provides basic header details with payload in hex and text format.
- **full:** Full alert mode, providing all possible information about the alert.
- **fast:** Fast mode, shows the alert message, timestamp, source, and destination ip along with port numbers.
- **none:** Disabling alerting.

In this section, only the "**console**" and "**cmg**" parameters provide alert information in the console. It is impossible to distinguish between the remaining alert modes via the terminal. Differences can be identified by looking at the generated logs.

At the end of this section, we will compare the "full", "fast", and "none" modes. Remember that these parameters don't provide console output, so we will continue to identify the differences through log formats.

IDS/IPS mode with parameter "-A console"

Console mode provides fast style alerts on the console screen. Start the Snort instance in **console alert mode (-A console)** with the following command: `sudo snort -c /etc/snort/snort.conf -A console`

Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will start generating alerts according to the provided ruleset defined in the configuration file.

```
Running in console alert mode

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -A console
Running in IDS mode

---= Initializing Snort ===
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
...
Commencing packet processing (pid=3743)
12/12-02:08:27.577495  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-02:08:27.577495  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-02:08:27.577495  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-02:08:27.609719  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
^C*** Caught Int-Signal
12/12-02:08:29.595898  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-02:08:29.595898  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-02:08:29.595898  [**] [1:384:5] ICMP PING [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.175.129 -> 142.250.187.110
=====
Run time for packet processing was 26.25844 seconds
Snort processed 88 packets.
```

IDS/IPS mode with parameter "-A cmg"

CMG mode provides basic header details, including the payload, in both hex and text formats. Start the Snort instance in **cmg alert mode (-A cmg)** with the following command: `sudo snort -c /etc/snort/snort.conf -A cmg`

Now run the traffic-generator script as **sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will start generating alerts according to the provided ruleset defined in the configuration file.

```
Running in CMG alert mode

, user@ubuntu$ sudo snort -c /etc/snort/snort.conf -A cmg
Running in IDS mode

---= Initializing Snort ===
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
...
Commencing packet processing (pid=3743)
12/12-02:23:56.944351  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc activity] [Priority: 3] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-02:23:56.944351  00:0C:29:A5:B7:A2 -> 00:50:56:E1:9B:9D type:0x800 len:0x62
192.168.175.129 -> 142.250.187.110 ICMP TTL:64 TOS:0x0 ID:10393 IpLen:20 DgmLen:84 DF
Type:8 Code:0 ID:4 Seq:1 ECHO
BC CD B5 61 00 00 00 00 CE 68 0E 00 00 00 00 00 00  ....a.....h.....
10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F  ....
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F  !"#$%&'()*+,-./
30 31 32 33 34 35 36 37  01234567

=====
```

Let's compare the console and CMG outputs before moving on to other alarm types. As you can see in the given outputs above, **console mode** provides basic header and rule information. **CMG mode** provides full packet details along with rule information.

IDS/IPS mode with parameter "-A fast"

Fast mode provides alert messages, timestamps, and source and destination IP addresses. **Remember, there is no console output in this mode.** Start the Snort instance in **fast alert mode (-A fast)** with the following command: `sudo snort -c /etc/snort/snort.conf -A fast`

Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will start generating alerts according to the provided ruleset defined in the configuration file.

```
Running in fast alert mode

, user@ubuntu$ sudo snort -c /etc/snort/snort.conf -A fast
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
...
Commencing packet processing (pid=3743)
=====
```

Let's check the alarm file.

```
1 12/12-02:19:35.373666  [**] [1:10000001:0] ICMP Packet found [**]
  [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
2 12/12-02:19:35.407782  [**] [1:10000001:0] ICMP Packet found [**]
  [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
3 12/12-02:19:36.409527  [**] [1:10000001:0] ICMP Packet found [**]
  [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
4 12/12-02:19:37.377857  [**] [1:10000001:0] ICMP Packet found [**]
  [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
5 12/12-02:19:37.409040  [**] [1:10000001:0] ICMP Packet found [**]
  [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
6 12/12-02:19:38.380477  [**] [1:10000001:0] ICMP Packet found [**]
  [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
```

As you can see in the picture above, fast style alerts contain summary information on the action, such as direction and alert header.

IDS/IPS mode with parameter "-A full"

Full alert mode provides all possible information about the alert. **Remember, there is no console output in this mode.** Start the Snort instance in **full alert mode (-A full)** with the following command: `sudo snort -c /etc/snort/snort.conf -A full`

Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will start generating alerts according to the provided ruleset defined in the configuration file.

```
Running in full alert mode

, user@ubuntu$ sudo snort -c /etc/snort/snort.conf -A full
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
...
Commencing packet processing (pid=3744)
=====
```

Let's check the alarm file.

```
1 [**] [1:10000001:0] ICMP Packet found [**]
2 [Priority: 0]
3 12/12-02:22:01.245268 192.168.175.129 -> 142.250.187.110
4 ICMP TTL:64 TOS:0x0 ID:61937 IpLen:20 DgmLen:84 DF
5 Type:8 Code:0 ID:3 Seq:1 ECHO
6
7 [**] [1:10000001:0] ICMP Packet found [**]
8 [Priority: 0]
9 12/12-02:22:01.278389 142.250.187.110 -> 192.168.175.129
10 ICMP TTL:128 TOS:0x0 ID:20624 IpLen:20 DgmLen:84
11 Type:0 Code:0 ID:3 Seq:1 ECHO REPLY
12
13 [**] [1:10000001:0] ICMP Packet found [**]
14 [Priority: 0]
15 12/12-02:22:02.247070 192.168.175.129 -> 142.250.187.110
16 ICMP TTL:64 TOS:0x0 ID:61976 IpLen:20 DgmLen:84 DF
17 Type:8 Code:0 ID:3 Seq:2 ECHO
18
19 [**] [1:10000001:0] ICMP Packet found [**]
20 [Priority: 0]
21 12/12-02:22:02.279396 142.250.187.110 -> 192.168.175.129
22 ICMP TTL:128 TOS:0x0 ID:20626 IpLen:20 DgmLen:84
23 Type:0 Code:0 ID:3 Seq:2 ECHO REPLY
```

As you can see in the picture above, full-style alerts contain all possible information on the action.

IDS/IPS mode with parameter "-A none"

Disable alerting. This mode doesn't create the alert file. However, it still logs the traffic and creates a log file in binary dump format. **Remember, there is no console output in this mode.** Start the Snort instance in **non-alert mode (-A none)** with the following command: `sudo snort -c /etc/snort/snort.conf -A none`

Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**. Once the traffic is generated, Snort will start generating alerts according to the provided ruleset defined in the configuration file.

```
Running in non-alert mode

, user@ubuntu$ sudo snort -c /etc/snort/snort.conf -A none
Running in IDS mode

--- Initializing Snort ---
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
...
Commencing packet processing (pid=3745)
=====
```

As you can see in the picture below, there is no alert file. Snort only generated the log file.

Name	Size
snort.log.1639307205	8.6 kB

IDS/IPS mode: "Using rule file without configuration file"

It is possible to run Snort with only rules, without a configuration file. Running Snort in this mode will help you test the user-created rules. However, this mode will provide less performance.

```
Running without a configuration file

, user@ubuntu$ sudo snort -c /etc/snort/rules/local.rules -A console
Running in IDS mode

12/12-12:13:29.167955  [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:29.200543  [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:30.169785  [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:30.201470  [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:31.172101  [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
^C*** Caught Int-Signal
```

IPS mode and dropping packets

Snort IPS mode is activated with the **-Q--daq afpacket** parameters. You can also activate this mode by editing the Snort.conf file. However, you don't need to edit the Snort.conf file in the scope of this room. Review the bonus task or snort manual for further information on daq and advanced configuration settings: `-Q --daq afpacket`

Activate the Data Acquisition (DAQ) modules and use the afpacket module to use Snort as an IPS: `-i eth0:eth1`

Identifying interfaces, note that Snort IPS requires at least two interfaces to work. Now run the traffic-generator script **as sudo** and start **ICMP/HTTP traffic**.

```
running IPS mode

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -q -Q --daq afpacket -i eth0:eth1 -A console
Running in IPS mode

12/18-07:40:01.527100  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.131 -> 192.168.175.2
12/18-07:40:01.552811  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 172.217.169.142 -> 192.168.1.18
12/18-07:40:01.566232  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.131 -> 192.168.175.2
12/18-07:40:02.517903  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.1.18 -> 172.217.169.142
12/18-07:40:02.550844  [Drop] [**] [1:1000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 172.217.169.142 -> 192.168.1.18
^C*** Caught Int-Signal
```

As you can see in the picture above, Snort blocked the packets this time. **We used the same rule with a different action (drop/reject)**. Remember, for the scope of this task, our point is the operating mode, not the rule.

Answer the questions below

Investigate the traffic with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l .
```

Execute the traffic generator script and choose "**TASK-7 Exercise**". Wait until the traffic stops, then stop the Snort instance. Now analyse the output summary and answer the question.

```
sudo ./traffic-generator.sh
```

What is the number of the detected HTTP GET methods?

✓ Correct Answer



You can practice the rest of the parameters by using the traffic-generator script.

✓ Correct Answer

Task 8 Operation Mode 4: PCAP Investigation



Let's investigate PCAPs with Snort

Capabilities of Snort are not limited to sniffing, logging and detecting/preventing the threats. PCAP read/investigate mode helps you work with pcap files. Once you have a pcap file and process it with Snort, you will receive default traffic statistics with alerts depending on your ruleset.

Reading a pcap without using any additional parameters we discussed before will only overview the packets and provide statistics about the file. In most cases, this is not very handy. We are investigating the pcap with Snort to benefit from the rules and speed up our investigation process by using the known patterns of threats.

Note that we are pretty close to starting to create rules. Therefore, you need to grasp the working mechanism of the Snort, learn the discussed parameters and begin combining the parameters for different purposes.

PCAP mode parameters are explained in the table below;

Parameter	Description
<code>-r / --pcap-single=</code>	Read a single pcap
<code>--pcap-list=""</code>	Read pcaps provided in command (space separated).
<code>--pcap-show</code>	Show pcap name on console during processing.

Investigating single PCAP with parameter "-r"

For test purposes, you can still test the default reading option with pcap by using the following command `snort -r icmp-test.pcap`

Let's investigate the pcap with our configuration file and see what will happen: `sudo snort -c /etc/snort/snort.conf -q -r icmp-test.pcap -A console -n 10`

If you don't remember the purpose of the parameters in the given command, please revisit previous tasks and come back again!

```
investigating single pcap file

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -q -r icmp-test.pcap -A console -n 10

12/12-12:13:29.167955  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:29.200543  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:30.169785  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:30.201470  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:31.172101  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:31.204104  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:32.174106  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:32.208683  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:33.176920  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:33.208359  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
```

Our ICMP rule got a hit! As you can see in the given output, snort identified the traffic and prompted the alerts according to our ruleset.

Investigating multiple PCAPs with parameter "--pcap-list"

Let's investigate multiple pcaps with our configuration file and see what will happen: `sudo snort -c /etc/snort/snort.conf -q --pcap-list="icmp-test.pcap http2.pcap" -A console -n 10`

```
investigating multiple pcap files

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -q --pcap-list="icmp-test.pcap http2.pcap" -A console

12/12-12:13:29.167955  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:29.200543  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:30.169785  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:30.201470  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:31.172101  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
...
12/12-12:13:31.204104  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:32.174106  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:32.208683  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:33.176920  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:33.208359  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
```

Our ICMP rule got a hit! As you can see in the given output, snort identified the traffic and prompted the alerts according to our ruleset.

Here is one point to notice: we've processed two pcaps, and there are lots of alerts, so it is impossible to match the alerts with provided pcaps without snort's help. We need to separate the pcap process to identify the source of the alerts.

Investigating multiple PCAPs with parameter "--pcap-show"

Let's investigate multiple pcaps, distinguish each one, and see what will happen. `sudo snort -c /etc/snort/snort.conf -q --pcap-list="icmp-test.pcap http2.pcap" -A console --pcap-show`

```
investigating multiple pcap files with pcap info

user@ubuntu$ sudo snort -c /etc/snort/snort.conf -q --pcap-list="icmp-test.pcap http2.pcap" -A console --pcap-show

Reading network traffic from "icmp-test.pcap" with snaplen = 1514
12/12-12:13:29.167955  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:29.200543  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:30.169785  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
...

Reading network traffic from "http2.pcap" with snaplen = 1514
12/12-12:13:35.213176  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
12/12-12:13:36.182950  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 192.168.175.129 -> 142.250.187.110
12/12-12:13:38.223470  [**] [1:10000001:0] ICMP Packet found [**] [Priority: 0] {ICMP} 142.250.187.110 -> 192.168.175.129
...
```

Our ICMP rule got a hit! As you can see in the given output, snort identified the traffic, distinguished each pcap file and prompted the alerts according to our ruleset.

Now, use the attached VM and navigate to the Task-Exercises/Exercise-Files/TASK-8 folder to answer the questions!

Answer the questions below

Investigate the **mx-1.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

✓ Correct Answer

Keep reading the output. How many TCP Segments are Queued?

✓ Correct Answer

Keep reading the output. How many "HTTP response headers" were extracted?

✓ Correct Answer

Investigate the **mx-1.pcap** file with the **second** configuration file.

```
sudo snort -c /etc/snort/snortv2.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

✓ Correct Answer

Investigate the **mx-2.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-2.pcap
```

What is the number of the generated alerts?

✓ Correct Answer



Keep reading the output. What is the number of the detected TCP packets?

✓ Correct Answer

Investigate the **mx-2.pcap** and **mx-3.pcap** files with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . --pcap-list="mx-2.pcap mx-3.pcap"
```

What is the number of the generated alerts?

✓ Correct Answer



Let's Learn Snort Rules

Understanding the Snort rule format is essential for any blue and purple teamer. The primary structure of the Snort rule is shown below;

Action	Protocol	Source IP	Source Port	Direction	Destination IP	Destination Port	Options
Alert	TCP	ANY	ANY	<>	ANY	ANY	Msg Reference Sid Rev
Drop	UDP						
Reject	ICMP						

Rule Header**Rule Options**

The following rule will generate an alert for each ICMP packet processed by snort;

```
alert icmp any any <> any any ( msg: "ICMP Packet found"; reference:CVE-XXXX; sid:1000001; rev:1; )
```

Rule Header

alert	Action type
icmp	Protocol type
any	Source IP
any	Source port
<>	Direction
any	Destination IP
any	Destination port

Rule Options

msg	Message
reference	Reference
sid	Rule id
rev	Revision information

Each rule should have a type of action, a protocol, source and destination IP addresses, source and destination port numbers, and an option. Remember, Snort is in passive mode by default. So most of the time, you will use Snort as an IDS. You will need to start **"inline mode" to turn on IPS mode**. However, before you begin using inline mode, it is essential to be familiar with Snort's features and rules.

The Snort rule structure is easy to understand but difficult to produce. You should be familiar with rule options and related details to create efficient rules. It is recommended to practice Snort rules and option details for different use cases.

We will cover the basic rule structure in this room and help you take a step into Snort rules. You can always advance your rule creation skills with different rule options by practising different use cases and studying rule option details in depth. We will focus on two actions: **"alert"** for IDS mode and **"reject"** for IPS mode.

Rules cannot be processed without a header. Rule options are "optional" parts. However, it is almost impossible to detect sophisticated attacks without using the rule options.

Action	There are several actions for rules. Ensure you understand the functionality and test it thoroughly before creating rules for live systems. The following are the most common actions: • alert: Generate an alert and log the packet. • log: Log the packet. • drop: Block and log the packet. • reject: Block the packet, log it, and terminate the packet session.
Protocol	The protocol parameter identifies the type of protocol that was filtered for the rule. Note that Snort2 supports only four protocol filters in the rules (IP, TCP, UDP, and ICMP). However, you can detect the application flows using port numbers and options. For instance, if you want to detect FTP traffic, you cannot use the FTP keyword in the protocol field; instead, you can filter FTP traffic by investigating TCP traffic on port 21.

IP and Port Numbers

These parameters identify the source and destination IP addresses, as well as the associated port numbers, filtered for the rule.

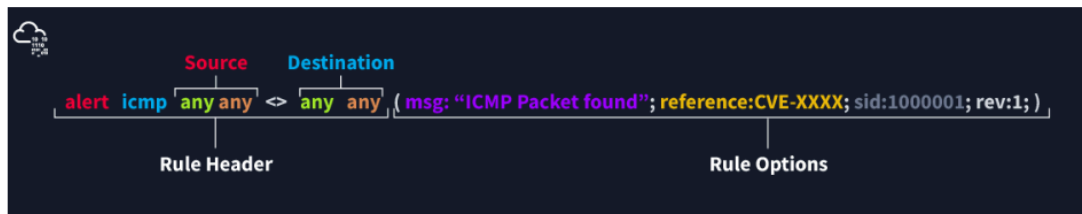
IP Filtering	<pre>alert icmp 192.168.1.56 any <=> any any (msg: "ICMP Packet From "; sid: 100001; rev:1;) This rule will create an alert for each ICMP packet originating from the 192.168.1.56 IP address.</pre>
Filter an IP range	<pre>alert icmp 192.168.1.0/24 any <=> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;) This rule will create an alert for each ICMP packet originating from the 192.168.1.0/24 subnet.</pre>
Filter multiple IP ranges	<pre>alert icmp [192.168.1.0/24, 10.1.1.0/24] any <=> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;) This rule will create an alert for each ICMP packet originating from the 192.168.1.0/24 and 10.1.1.0/24 subnets.</pre>
Exclude IP addresses/ranges.	The "negation operator" is used to exclude specific addresses and ports. The negation operator is indicated with "!". <pre>alert icmp !192.168.1.0/24 any <=> any any (msg: "ICMP Packet Found"; sid: 100001; rev:1;) This rule will create an alert for each ICMP packet not originating from the 192.168.1.0/24 subnet.</pre>
Port Filtering	<pre>alert tcp any any <=> any 21 (msg: "FTP Port 21 Command Activity Detected"; sid: 100001; rev:1;) This rule will create an alert for each TCP packet sent to port 21.</pre>
Exclude a specific port	<pre>alert tcp any any <=> any !21 (msg: "Traffic Activity Without FTP Port 21 Command Channel"; sid: 100001; rev:1;) This rule will create an alert for each TCP packet not sent to port 21.</pre>
Filter a port range (Type 1)	<pre>alert tcp any any <=> any 1:1024 (msg: "TCP 1-1024 System Port Activity"; sid: 100001; rev:1;) This rule will create an alert for each TCP packet sent to ports between 1 and 1024.</pre>
Filter a port range (Type 2)	<pre>alert tcp any any <=> any:1024 (msg: "TCP 0-1024 System Port Activity"; sid: 100001; rev:1;) This rule will create an alert for each TCP packet sent to ports 1 through 1024.</pre>
Filter a port range (Type 3)	<pre>alert tcp any any <=> any 1025: (msg: "TCP Non-System Port Activity"; sid: 100001; rev:1;) This rule will create an alert for each TCP packet sent to a source port that is higher than or equal to 1025.</pre>
Filter a port range (Type 4)	<pre>alert tcp any any <=> any [21,23] (msg: "FTP and Telnet Port 21-23 Activity Detected"; sid: 100001; rev:1;) This rule will create an alert for each TCP packet sent to ports 21 and 23.</pre>

Direction

The direction operator indicates the traffic flow to be filtered by Snort. The left side of the rule shows the source, and the right side shows the destination.

- -> Source to destination flow.
- <> Bidirectional flow

Note that there is no "<" operator in Snort.



There are three main rule options in Snort.

- **General Rule Options** - Fundamental rule options for Snort.
- **Payload Rule Options** - Rule options that help to investigate the payload data. These options are helpful to detect specific payload patterns.
- **Non-Payload Rule Options** - Rule options that focus on non-payload data. These options will help create specific patterns and identify network issues.

General Rule Options

Msg	The message field is a basic prompt and quick identifier of the rule. Once the rule is triggered, the message field will appear in the console or log. Usually, the message part is a one-liner that summarises the event.
Sid	Snort rule <u>IDs</u> (SIDs) come with a predefined scope, and each rule must have a SID in the proper format. There are three different scopes for SIDs shown below. • <100: Reserved rules • 100-999,999: Rules came with the build. • >=1,000,000: Rules created by user. Briefly, the rules we will create should have a side greater than 100.000.000. Another important point is that SIDs should not overlap, and each ID must be unique.
Reference	Each rule can have additional information or a reference to explain the purpose of the rule or threat pattern. That could be a Common Vulnerabilities and Exposures (CVE) ID or external data. Having references for the rules will always help analysts during the alert and incident investigation.
Rev	Snort rules can be modified and updated for performance and efficiency issues. The revision option enables analysts to access revision information for each rule. Therefore, it will be easy to understand rule improvements. Each rule has its unique rev number, and there is no auto-backup feature on the rule history. Analysts should keep their own rule history. The 'Rev' option is only an indicator of how many times the rule has undergone revisions. alert icmp any any <> any any (msg: "ICMP Packet Found"; sid: 100001; reference:cve,CVE-XXXX; rev:1;)

Payload Detection Rule Options

Content	Payload data. It matches specific payload data by ASCII, HEX, or both. This option can be used multiple times within a single rule. However, the more specific pattern match features you create, the more time it takes to investigate a packet. The following rules will create an alert for each HTTP packet containing the keyword "GET". This rule option is case sensitive! • ASCII mode - alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET"; sid: 100001; rev:1;) • HEX mode - alert tcp any any <> any 80 (msg: "GET Request Found"; content:" 47 45 54 "; sid: 100001; rev:1;)
Nocase	Disabling case sensitivity. Used for enhancing the content searches. alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET"; nocase; sid: 100001; rev:1;)
Fast_pattern	Prioritise content search to speed up the payload search operation. By default, Snort uses the largest content and evaluates it against the rules. The "fast_pattern" option helps you select the initial packet match with a specific value for further investigation. This option always works case-insensitively and can be used once per rule. Note that this option is required when using multiple "content" options. The following rule has two content options, and the fast_pattern option instructs Snort to use the first content option (in this case, "GET") for the initial packet match. alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET"; fast_pattern; content:"www"; sid:100001; rev:1;)

Non-Payload Detection Rule Options

There are rule options that focus on non-payload data. These options will help create specific patterns and identify network issues.

ID	Filtering the IP ID field. alert tcp any any <> any any (msg: "ID TEST"; id:123456; sid: 100001; rev:1;)
Flags	Filtering the TCP flags. • F - FIN • S - SYN • R - RST • P - PSH • A - ACK • U - URG alert tcp any any <> any any (msg: "FLAG TEST"; flags:S; sid: 100001; rev:1;)
Dsize	Filtering the packet payload size. • dsize:min<>max; • dsize:>100 • dsize:<100 alert ip any any <> any any (msg: "SEQ TEST"; dsize:100<>300; sid: 100001; rev:1;)
Sameip	Filtering the source and destination IP addresses for duplication. alert ip any any <> any any (msg: "SAME-IP TEST"; sameip; sid: 100001; rev:1;)

Remember, once you create a rule, it is a local rule and should be in your "local.rules" file. This file is located under "/etc/snort/rules/local.rules". A quick reminder on how to edit your local rules is shown below.

```
modifying the local rules

user@ubuntu$ sudo gedit /etc/snort/rules/local.rules
```

That is your "local.rules" file.

```
Open local.rules /etc/snort/rules
1
2 # -----
3 # LOCAL RULES
4 # -----
5 # This file intentionally does not come with signatures. Put your local
6 # additions here.
7
8 alert icmp any any <> any any (msg: "ICMP Packet Found"; sid: 1000001; rev:1;)
9
10
```

Note that some default rules are activated with the Snort instance. These rules are deactivated to manage your rules and improve your exercise experience. For further information, please refer to the TASK-10 or [Snort manual](#).

By this point, we have covered the primary structure of the Snort rules. Understanding and practicing the fundamentals is suggested before creating advanced rules and using additional options.

Wow! We have covered the fundamentals of the Snort rules! Now, use the attached VM and navigate to the `Task-Exercises/Exercise-Files/TASK-9` folder to answer the questions! Note that you can use the following command to create logs in the current directory: -l

Answer the questions below

Use "task9.pcap". Write a rule to filter IP ID "35369" and run it against the given pcap file. What is the request name of the detected packet? You may use this command: "snort -c local.rules -A full -l . -r task9.pcap"

Timestamp Request

✓ Correct Answer



Clear the previous alert file and comment out the old rules. Create a rule to filter packets with **Syn** flag and run it against the given pcap file. What is the number of detected packets?

1

✓ Correct Answer

Clear the previous alert file and comment out the old rules. Write a rule to filter packets with **Push-Ack** flags and run it against the given pcap file. What is the number of detected packets?

216

✓ Correct Answer

Clear the previous alert file and comment out the old rules. Create a rule to filter **UDP** packets with the same source and destination IP and run it against the given pcap file. What is the number of packets that show the same source and destination address?

7

✓ Correct Answer

Case Example - An analyst modified an existing rule successfully. Which rule option must the analyst change after the implementation?

rev

✓ Correct Answer

Points to Remember

Main Components of Snort

- Packet Decoder - Packet collector component of Snort. It collects and prepares the packets for pre-processing.
- Pre-processors - A component that arranges and modifies the packets for the detection engine.
- Detection Engine - The primary component that processes, dissects, and analyzes the packets by applying the rules.
- Logging and Alerting - Log and alert generation component.
- Outputs and Plugins - Output integration modules (i.e., alerts to syslog/mysql) and additional plugins (rule management detection plugins) support is done with this component.

Snort Rules

There are three types of rules available for Snort, which are explained below:

- Community Rules - Free ruleset under the GPLv2. Publicly accessible, no need for registration.
- Registered Rules - Free ruleset (requires registration). This ruleset contains subscriber rules with 30 30-day delay.
- Subscriber Rules (Paid) - Paid ruleset (requires subscription). This ruleset is the main ruleset and is updated twice a week (Tuesdays and Thursdays).

You can download and read more on the rules [here](#).

Note: After installing Snort2, it automatically creates the required directories and files. However, if you want to use the community or paid rules, you need to indicate each rule in Snort.conf file.

Since it is a long, all-in-one configuration file, editing it without causing misconfiguration is troublesome for some users. That is why Snort has several rule updating modules and integration tools. To sum up, never replace your configured Snort configuration files; instead, edit your configuration files manually or update your rules with additional tools and modules to prevent failures, crashes, or missing features.

- snort.conf: Main configuration file.
- local.rules: User-generated rules file.

Let's start with an overview of the main configuration file (snort.conf). `sudo gedit /etc/snort/snort.conf`.

Navigate to the "Step #1: Set the network variables" section.

This section manages the scope of the detection and rule paths.

TAG NAME	INFO	EXAMPLE
HOME_NET	That is where we are protecting.	'any' OR '192.168.1.1/24'
EXTERNAL_NET	This field represents the external network, so we need to keep it set to 'any' or '!'\$HOME_NET'.	'any' OR '!'\$HOME_NET'
RULE_PATH	Hardcoded rule path.	/etc/snort/rules
SO_RULE_PATH	These rules are accompanied by registered and subscriber rules.	\$RULE_PATH/so_rules
PREPROC_RULE_PATH	These rules are accompanied by registered and subscriber rules.	\$RULE_PATH/plugin_rules

Navigate to the "Step #2: Configure the decoder" section.

In this section, you manage the `IPS` mode of Snort. The single-node installation model `IPS` model works best with "afpacket" mode. You can enable this mode and run Snort in `IPS`.

TAG NAME	INFO	EXAMPLE
#config daq	<code>IPS</code> mode selection.	afpacket
#config daq_mode	Activating the inline mode	inline
#config logdir	Hardcoded default log path.	/var/logs/snort

Data Acquisition Modules (DAQs) are specialized libraries used for packet I/O, providing flexibility in processing packets. It is possible to select the DAQ type and mode for different purposes.

There are six DAQ modules available in Snort:

- PCAP: Default mode, known as Sniffer mode.
- Afpacket: Inline mode, known as IPS mode.
- Ipq: Inline mode on Linux by using Netfilter. It replaces the snort_inline patch.
- Nfq: Inline mode on Linux.
- Ipfw: Inline on OpenBSD and FreeBSD by using divert sockets, with the pf and ipfw firewalls.
- Dump: Testing mode of inline and normalisation.

The most popular modes are the default (pcap) and inline/IPS (Afpacket).

Navigate to the "Step #6: Configure output plugins" section.

This section manages the outputs of IDS/IPS actions, including logging and alerting format details. The default action prompts everything in the console application, so configuring this part will help you use Snort more efficiently.

Navigate to the "Step #7: Customise your ruleset" section.

TAG NAME	INFO	EXAMPLE
# site-specific rules	Hardcoded local and user-generated rules path.	include \$RULE_PATH/local.rules
#include \$RULE_PATH/	Hardcoded default/downloaded rules path.	include \$RULE_PATH/rulename

Note that "#" is a commenting operator. You should uncomment a line to activate it.

Answer the questions below

Read the task above.

No answer needed

✓ Correct Answer

Task 11 ✓ Conclusion

📄

↺

⬆

In this room, we covered Snort, its definition, operation, and how to create and use rules to investigate threats.

Download Task Files

- Understanding and practising the fundamentals is crucial before creating advanced rules and using additional options.
- Do not create complex rules at once; try adding options step by step to easily notice possible syntax errors or other problems.
- Do not reinvent the wheel; use or modify/enhance it if there is a smooth rule.
- Take a backup of the configuration files before making any changes.
- Never delete a rule that works properly. Comment on it if you don't need it.
- Test newly created rules before migrating them to production.

Now, we invite you to complete a series of Snort challenges, starting from [Snort Challenge - The Basics](#).

A great way to quickly recall snort rules and commands is to download and refer to the TryHackMe snort cheatsheet.

Answer the questions below

Read the task above.

No answer needed

✓ Correct Answer

Result

Snort was successfully used as an IDS/IPS to detect and prevent network threats, demonstrating real-time network security monitoring in a controlled environment.